University of Central Missouri

Department of Computer Science & Cybersecurity

CS5760 Natural Language Processing

Spring 2026

Homework 2.

Student name: Janani Govuri

700773302

Submission Requirements:

- Once finished your assignment push your source code to your repo (GitHub) and explain the work through the ReadMe file properly. Make sure you add your student info in the ReadMe file.
- Submit your GitHub link on the Bright Space.
- Comment your code appropriately ***IMPORTANT.***
- Any submission after provided deadline is considered as a late submission.

Part I. Writing Calculation

Q1. Worked Example Document Classification (based on slide: Test document "predictable no fun")
Using the smoothed likelihoods and priors from Q2, compute the probability scores for the document *"predictable no fun"* under both the positive and negative classes.

Tasks:

1. Show each step of the multiplication.
2. Which class should the system assign to this document?

Q2. Harms of Classification (based on slide: Avoiding Harms in Classification)

Tasks:

1. Define representational harm and explain how the Kiritchenko & Mohammad (2018) study demonstrates this type of harm.
2. What is one risk of censorship in toxicity classification systems (based on Dixon et al. 2018, Oliva et al. 2021)?
3. Give one reason why classifiers may perform worse on African American English or Indian English, even though they are varieties of English.

Q3. Bigram Probabilities and the Zero-Probability Problem

You are given the following bigram counts from a small training corpus:

Previous word	Next words (with counts)
<s>	I: 2 , deep: 1
I	love: 2
love	NLP: 1 , deep: 1
deep	learning: 2
learning	</s>: 1 , is: 1
NLP	</s>: 1
is	fun: 1
fun	</s>: 1
ate	lunch: 6 , dinner: 3 , a: 2 , the: 1

Tasks:

1. Bigram Sentence Probabilities
   Using maximum likelihood estimation (MLE):

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

   - Compute the probability of sentence S1: <s> I love NLP </s>.
   - Compute the probability of sentence S2: <s> I love deep learning </s>.
   - Which sentence is more probable under the bigram model?

2. Zero-Probability Problem
   Using the same table, compute:
   - P(noodle|ate) with MLE.
   - Explain why this probability creates problems when computing sentence probabilities or perplexity.

- Apply Laplace smoothing (Add-1) to recompute P(noodle|ate). Assume the vocabulary size is 10 and total count after "ate" is 12.

Q4. Backoff Model (based on "Activity: <s> I like cats … You like dogs" slide)

Training corpus:

<s> I like cats </s>
<s> I like dogs </s>
<s> You like cats </s>

Counts:

- I like = 2
- You like = 1
- like cats = 2
- like dogs = 1
- cats </s> = 2
- dogs </s> = 1

Tasks:

1. Compute P(cats|I,like).
2. Compute P(dogs|You,like) using trigram → bigram backoff.
3. Explain why backoff is necessary in this example.

Q5: Evaluation Metrics from a Multi-Class Confusion Matrix

The system classified 90 animals into Cat, Dog, or Rabbit. The results are shown below:

System \ Gold	**Cat**	**Dog**	**Rabbit**
Cat	5	10	5
Dog	15	20	10
Rabbit	0	15	10

Tasks:

1. Per-Class Metrics
   - Compute precision and recall for each class (Cat, Dog, Rabbit).
2. Macro vs. Micro Averaging
   - Compute the macro-averaged precision and recall.
   - Compute the micro-averaged precision and recall.

- Briefly explain the difference in interpretation between macro and micro averaging.

3. **Programming Implementation**

   Write Python code that:

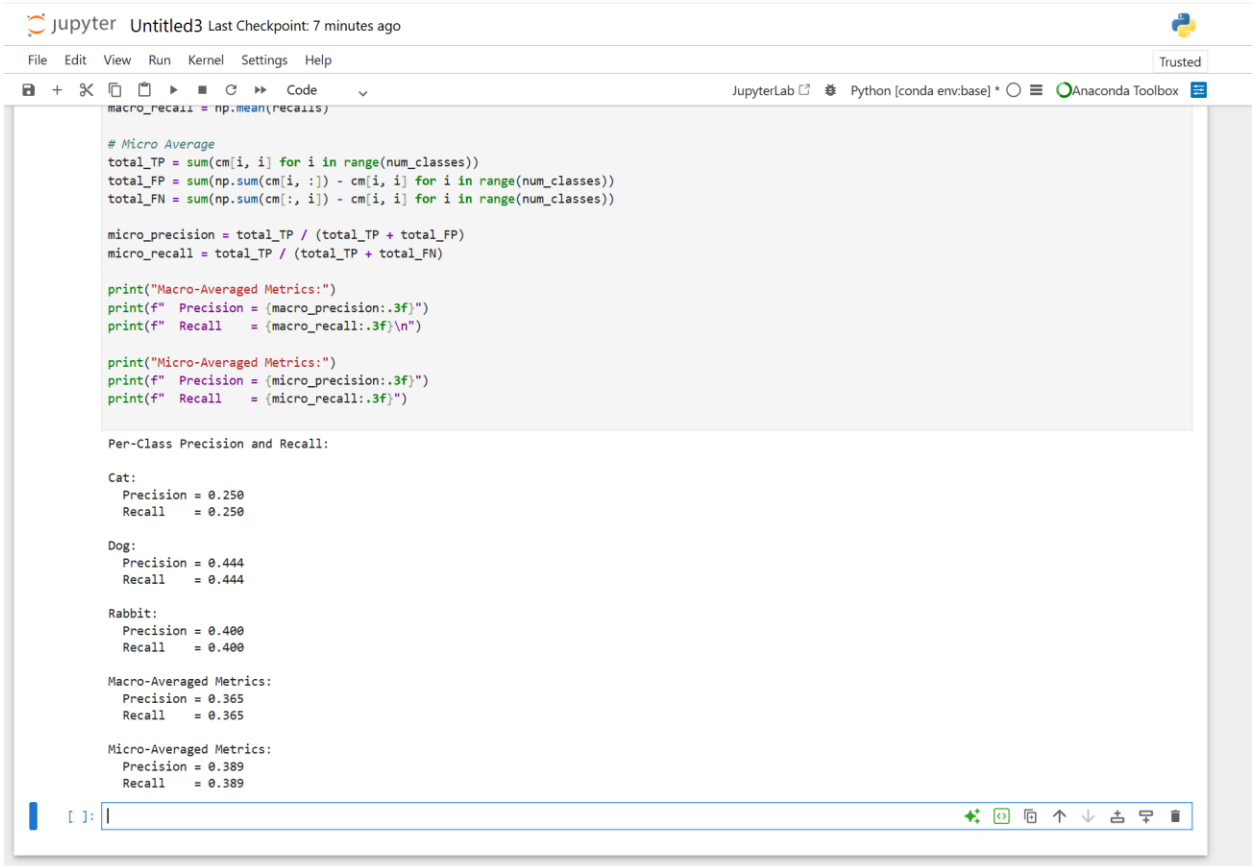
   1. Accepts the confusion matrix above as input.
   2. Computes per-class precision and recall.
   3. Computes macro-averaged and micro-averaged precision and recall.
   4. Prints all results clearly.

jupyter Untitled3 Last Checkpoint: 7 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

[2]:
```python
import numpy as np

# Confusion Matrix
# Rows = Predicted (System)
# Columns = Actual (Gold)
cm = np.array([
    [5, 10, 5],    # Predicted Cat
    [15, 20, 10],  # Predicted Dog
    [0, 15, 10]    # Predicted Rabbit
])

classes = ["Cat", "Dog", "Rabbit"]

num_classes = len(classes)

precisions = []
recalls = []

print("Per-Class Precision and Recall:\n")

for i in range(num_classes):
    TP = cm[i, i]
    FP = np.sum(cm[i, :]) - TP
    FN = np.sum(cm[:, i]) - TP

    precision = TP / (TP + FP)
    recall = TP / (TP + FN)

    precisions.append(precision)
    recalls.append(recall)

    print(f"{classes[i]}:")
    print(f"  Precision = {precision:.3f}")
    print(f"  Recall    = {recall:.3f}\n")

# Macro Average
macro_precision = np.mean(precisions)
macro_recall = np.mean(recalls)

# Micro Average
total_TP = sum(cm[i, i] for i in range(num_classes))
total_FP = sum(np.sum(cm[i, :]) - cm[i, i] for i in range(num_classes))
total_FN = sum(np.sum(cm[:, i]) - cm[i, i] for i in range(num_classes))
```



```
macro_recall = np.mean(recalls)

# Micro Average
total_TP = sum(cm[i, i] for i in range(num_classes))
total_FP = sum(np.sum(cm[i, :]) - cm[i, i] for i in range(num_classes))
total_FN = sum(np.sum(cm[:, i]) - cm[i, i] for i in range(num_classes))

micro_precision = total_TP / (total_TP + total_FP)
micro_recall = total_TP / (total_TP + total_FN)

print("Macro-Averaged Metrics:")
print(f"  Precision = {macro_precision:.3f}")
print(f"  Recall    = {macro_recall:.3f}\n")

print("Micro-Averaged Metrics:")
print(f"  Precision = {micro_precision:.3f}")
print(f"  Recall    = {micro_recall:.3f}")
```

```
Per-Class Precision and Recall:

Cat:
  Precision = 0.250
  Recall    = 0.250

Dog:
  Precision = 0.444
  Recall    = 0.444

Rabbit:
  Precision = 0.400
  Recall    = 0.400

Macro-Averaged Metrics:
  Precision = 0.365
  Recall    = 0.365

Micro-Averaged Metrics:
  Precision = 0.389
  Recall    = 0.389
```

Part II. Programming

Q1. Programming: Bigram Language Model Implementation (based on "Activity: I love NLP corpus" slide)

Tasks:

Write a Python program to:

1. Read the training corpus:
2. <s> I love NLP </s>
3. <s> I love deep learning </s>
4. <s> deep learning is fun </s>
5. Compute unigram and bigram counts.
6. Estimate bigram probabilities using MLE.
7. Implement a function that calculates the probability of any given sentence.
8. Test your function on both sentences:
    - <s> I love NLP </s>
    - <s> I love deep learning </s>
9. Print which sentence the model prefers and why.

jupyter Untitled3 Last Checkpoint: 2 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

[1]:
```python
from collections import defaultdict

corpus = [
    ["<s>","I","love","NLP","</s>"],
    ["<s>","I","love","deep","learning","</s>"],
    ["<s>","deep","learning","is","fun","</s>"]
]

unigrams = defaultdict(int)
bigrams = defaultdict(int)

for sent in corpus:
    for i in range(len(sent)):
        unigrams[sent[i]] += 1
        if i > 0:
            bigrams[(sent[i-1],sent[i])] += 1

def bigram_prob(w1,w2):
    return bigrams[(w1,w2)]/unigrams[w1]

def sentence_prob(sentence):
    prob = 1
    for i in range(1,len(sentence)):
        prob *= bigram_prob(sentence[i-1],sentence[i])
    return prob

s1 = ["<s>","I","love","NLP","</s>"]
s2 = ["<s>","I","love","deep","learning","</s>"]

p1 = sentence_prob(s1)
p2 = sentence_prob(s2)

print("S1 probability:", p1)
print("S2 probability:", p2)

if p1 > p2:
    print("Model prefers S1")
else:
    print("Model prefers S2")
```

```
S1 probability: 0.3333333333333333
S2 probability: 0.16666666666666666
Model prefers S1
```